

Web SDK (Software Development Kit)

A **Web SDK** is a collection of tools and resources designed to help you seamlessly integrate specific features into your website or web application with minimal coding. Without a Web SDK, the integration process can be significantly more complex, as it requires building and testing each part of the solution from scratch. By using a Web SDK, you can simplify the process by configuring it to meet your particular workflow and requirements.

Ways to Implement a Web SDK:

1) Using Vanilla JavaScript

This approach involves integrating the Web SDK directly into your web application using plain JavaScript. It allows you to add the necessary features without relying on any additional frameworks or dependencies.

2) Using npm (Node Package Manager)

If you are working with a specific framework (such as React, Angular, or Vue), you can install the Web SDK using npm. This method is especially useful if your project already uses npm to manage dependencies. By installing the SDK through npm, you can easily integrate and update the Web SDK within your framework.

Using Vanilla JavaScript

Integrating a **Web SDK** into a web application using **vanilla JavaScript** means creating a set of reusable JavaScript modules that expose functions, classes, or utilities for use by other applications. This approach doesn't rely on any frameworks or external dependencies like Angular, React, or Vue. It works with basic HTML, CSS, and JavaScript.

We can create a web SDK using JavaScript in two ways:

- 1) ES Module Approach
- 2) Traditional (non-modular) Approach

Traditional (non-modular) Approach

In the traditional, non-modular approach, the SDK is written as a single script that contains all the logic. Typically, a global object or function is used to encapsulate the SDK's functionality. While this approach is simple and straightforward, it can become difficult to maintain as the codebase grows.

Advantages of the Non-Modular Approach:

1. **Simplicity:** Writing and understanding the SDK is easier, especially for smaller projects or quick prototypes.
2. **Fewer Files:** You don't need to manage multiple files for different pieces of functionality.

Disadvantages of the Non-Modular Approach:

1. **Scalability:** As the SDK expands, managing and maintaining a large amount of code in a single file becomes increasingly challenging.
2. **Namespace Pollution:** There's a risk of global namespace pollution, where multiple scripts may accidentally overwrite each other's functions or variables.

3. **Difficult to Debug:** If a bug arises, tracking down the cause in a large, monolithic script can be time-consuming and complicated.

Steps to Build and Bundle a Non-Modular SDK:

1. **Write the SDK Code:**
 - a. Implement the SDK functionality in a single JavaScript file.
 - b. Use global objects or functions to encapsulate the SDK methods.
2. **Test the SDK Locally:**
 - a. Create a simple HTML file to include and test the SDK.
 - b. Ensure that the SDK functions correctly when used in the browser.
3. **Bundle the SDK (Optional):**
 - a. If you want to optimize the code (e.g., minify, bundle), use bundlers like **Webpack** or **Parcel**.
 - b. Minify the code to reduce file size using tools like **Terser**.
4. **Host the SDK:**
 - a. Upload the SDK to a server, a cloud platform, or a **CDN** (Content Delivery Network).
 - b. Ensure the SDK is accessible via a public URL for easy inclusion in user projects.
5. **Provide Documentation:**
 - a. Create documentation that explains how to install and use the SDK.
 - b. Include code examples and a reference guide for the available methods.
6. **Versioning and Updates:**
 - a. Use version control to manage updates and improvements to the SDK.
 - b. When releasing new versions, update the version number and ensure users can access the latest version.

Greeting SDK: A Personalized Multilingual Web SDK for non-modular Approach

- The greet method now requires both name and language to function properly.
- If either parameter is missing or invalid, an error message will be displayed in the console.

SDK CODE:

```
(function(global) {  
  // Greeting class constructor  
  function Greeting() {
```

```

    this.version = '1.0';
    console.log('Web SDK initialized.');
```

```
// Greet method
```

```
Greeting.prototype.greet = function(name) {
    if (!name || typeof name !== 'string') {
        console.error('Invalid name provided');
        return null;
    }
}
```

```

    const greeting = `Hello, ${name}! Welcome to the Web SDK.`;
    console.log(greeting);
    return greeting;
};
```

```
// Advanced method: Multilingual greeting
```

```
Greeting.prototype.greetInLanguage = function(name, language) {
    const greetings = {
        'en': `Hello, ${name}!`,
        'es': `¡Hola, ${name}!`,
        'fr': `Bonjour, ${name}!`,
        'de': `Hallo, ${name}!`,
        'default': `Hello, ${name}!`
    };
};
```

```

    const greeting = greetings[language] || greetings['default'];
    console.log(greeting);
    return greeting;
};
```

```

// Expose the Greeting constructor to the global scope
global.Greeting = Greeting;
```

```
})(typeof window !== 'undefined' ? window : global);
```

IMPLEMENTATION OF GREETING SDK (TRADITIONAL WAY)

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Greeting SDK Test</title>
</head>
<body>
    <h1>Greeting SDK Demo</h1>
    <input type="text" id="nameInput" placeholder="Enter your name">
    <select id="languageSelect">
```

```

    <option value="en">English</option>
    <option value="es">Spanish</option>
    <option value="fr">French</option>
    <option value="de">German</option>
  </select>
  <button onclick="greetUser()">Greet</button>
  <div id="output"></div>

  // path of sdk
  <script src="https://web-sdk-demo/greeting-sdk.js"></script>
  <script>
    const greetingSdk = new Greeting();

    function greetUser() {
      const nameInput = document.getElementById('nameInput');
      const languageSelect = document.getElementById('languageSelect');
      const outputDiv = document.getElementById('output');

      if (nameInput.value) {
        // Use the greetInLanguage method
        const greeting = greetingSdk.greetInLanguage(
          nameInput.value,
          languageSelect.value
        );

        outputDiv.textContent = greeting;
      } else {
        outputDiv.textContent = 'Please enter a name';
      }
    }
  </script>
</body>
</html>

```

ES Module Approach (Modern Approach)

In the modular approach, you break the SDK into smaller, reusable, and maintainable pieces or modules. Each module is responsible for a specific functionality, making the SDK more organized, easier to maintain, and scalable.

Key Features:

- **Separation of Concerns:** Each module handles a specific responsibility (e.g., authentication, API requests, user interaction, etc.), which makes the code more maintainable and testable.
- **Reusability:** Since each module is self-contained, you can reuse them across multiple projects or parts of the SDK.
- **Dependency Management:** It is easier to handle dependencies between modules, and tools like **Webpack**, **Rollup**, or **Parcel** can bundle everything into a single file while managing dependencies.
- **Tree Shaking:** Modern bundlers support tree shaking, which removes unused code, making the SDK lighter and more efficient.
- **Versioning:** Each module can be versioned separately, so you can update individual modules without affecting the whole SDK.
- **Compatibility:** Works well with modern JavaScript tools and frameworks, like ES6 modules, React, or Angular.

Steps to Build and Bundle the ES Module SDK:

1. **Create Multiple Modules:** Break your SDK into distinct JavaScript files, each handling a specific function or feature.
2. **Use ES6 Modules:** Use import and export statements to manage dependencies between modules.
3. **Bundling:** Use bundlers like **Webpack**, **Rollup**, or **Parcel** to bundle these modules into a single file, making it easy to include in your project.
4. **Minification:** Minify your bundled JavaScript to reduce the file size. Tools like **Terser** can help with this.
5. **Publish:** You can publish your SDK on platforms like **npm** for easy installation or host it via a CDN for quicker access.
6. **Documentation:** Document each module separately, so users can integrate only the specific parts they need.

Greeting SDK: A Personalized Multilingual Web SDK for ES Module Approach

- The greet method now requires both name and language to function properly.
- If either parameter is missing or invalid, an error message will be displayed in the console.

SDK CODE:

```
export class Greeting {
  constructor() {
    this.version = '1.0';
    console.log('Web SDK initialized.');
```

```

greet(name) {
  if (!name || typeof name !== 'string') {
    console.error('Invalid name provided');
    return 'Invalid name provided';
  }

  const greeting = `Hello, ${name}! Welcome to the Web SDK.`;
  console.log(greeting);
  return greeting;
}

greetInLanguage(name, language) {
  if (!name || typeof name !== 'string') {
    console.error('Invalid name provided');
    return 'Invalid name provided';
  }

  const greetings = {
    'en': `Hello, ${name}!`,
    'es': `¡Hola, ${name}!`,
    'fr': `Bonjour, ${name}!`,
    'de': `Hallo, ${name}!`,
    'default': `Hello, ${name}!`
  };

  const greeting = greetings[language] || greetings['default'];
  console.log(greeting);
  return greeting;
}
}

```

IMPLEMENTATION OF GREETING SDK (ES MODULE WAY)

```

<!DOCTYPE html>
<html>
<head>
  <title>Greeting SDK Example (ES Module)</title>
</head>
<body>
  <h1>Greeting SDK Example</h1>
  <input type="text" id="nameInput" placeholder="Enter name">
  <select id="languageSelect">
    <option value="en">English</option>

```

```

        <option value="es">Spanish</option>
        <option value="fr">French</option>
        <option value="de">German</option>
    </select>
    <button id="greetButton">Greet</button>
    <p id="greetingOutput"></p>

    <script type="module">
        // Adjust path if needed
        import { Greeting } from 'https://web-sdk-demo/greeting-sdk.js';

        const greeter = new Greeting();

        document.getElementById('greetButton').addEventListener('click',
function() {
    const name = document.getElementById('nameInput').value;
    const language = document.getElementById('languageSelect').value;

    const greeting = greeter.greetInLanguage(name, language);
    document.getElementById('greetingOutput').textContent = greeting;
});
    </script>
</body>
</html>

```

How to make bundle of js ?

To create a bundle of JavaScript (JS), you need to use a **bundler** tool. Bundling is the process of combining multiple JavaScript files into a single (or a few) file(s) that can be included in a webpage or application. This is especially useful for managing dependencies, optimizing load times, and ensuring compatibility across environments.

Common Bundlers:

- Webpack
- Rollup
- Parcel
- ESBuild

1. Using Webpack

Step 1: Install Webpack

To get started with Webpack, you'll need to install it along with the Webpack CLI and Babel for transpiling your JavaScript code (if needed).

Command to install Webpack:

```
npm install --save-dev webpack webpack-cli babel-loader @babel/core @babel/preset-env
```

Step 2: Create a `webpack.config.js` File

Next, create a `webpack.config.js` file to configure how Webpack bundles your code.

`Webpack.config.js`

```
const path = require('path');

module.exports = {
  entry: './greeting-sdk.js',
  output: {
    filename: 'greeting-sdk-webpack.bundle.js', // The output bundle
    path: path.resolve(__dirname, 'dist'), // Output directory
    library: {
      name: 'Greeting',
      type: 'umd',
    },
    globalObject: 'this',
  },
  mode: 'development', // Set to 'development' for easier debugging
  devtool: 'source-map', // or 'eval-source-map' for better debugging
};
```

Step 3: Add Build Script to `package.json`

To make it easy to build your bundle, add a script in your `package.json` file:

```
"scripts": {
  "build": "webpack --config webpack.config.js"
},
```

Step 4: Build the Bundle

Now, run the build script to create the bundle:


```
npm run build
```

This will create a bundle.js file in the dist/ folder.

2. Using Rollup

Step 1: Install Rollup

To use Rollup for bundling, install Rollup along with the necessary plugins. Here's how you can install Rollup and the Babel plugin:

```
npm install --save-dev rollup rollup-plugin-babel @babel/preset-env
```

Step 2: Create a rollup.config.js File

Create a rollup.config.js file to define how Rollup will bundle your JavaScript code.

rollup.config.js:

```
// rollup.config.js
import resolve from '@rollup/plugin-node-resolve';

export default {
  input: 'greeting-sdk.js',
  output: {
    file: 'dist/greeting-sdk-rollup.bundle.js', // Output file path
    format: 'iife', // Immediately Invoked Function Expression
    name: 'GreetingSDK', // Global variable name
  },
  plugins: [
    resolve(), // Use the node-resolve plugin
  ],
};
```

Step 3: Build the Bundle

Run the following command to create the bundle:

```
npx rollup -c
```

This will create a bundle.js file in the dist/ folder.

3. Using Parcel (Zero Configuration)

Step 1: Install Parcel

Parcel is a zero-config bundler that doesn't require any configuration file for most use cases.

To get started, install Parcel:

```
npm install --save-dev parcel
```

Step 2: Add Build Script to package.json

Add a build script in your package.json file:

```
{
  "scripts": {
    "build": "parcel build src/index.js --out-dir dist"
  }
}
```

Step 3: Build the Bundle

Run the following command to create the bundle:

```
npm run build
```

Parcel automatically bundles your src/index.js file and outputs it into the dist/ folder as index.js.